

Introduction to Java EE and Enterprise Applications

Enterprise Application Development

Outline

- Enterprise Applications
- What is Java EE
- EJBs
- A brief technical look at EJBs
- Java EE APIs
- Additional Section : Enterprise Application Integration.

Enterprise Applications

- Applications designed to solve problems encountered by large enterprises
- Used by large corporations, governments, etc.
- Useful for even small organizations in an increasingly networked world

Large-scale • Scalable

Reliable • Secure

Enterp. Applications Requirements

- **Large :**A multi-user, multi-developer, multi-machine, multi-component application that can manipulate massive data and utilize extensive parallel processing, network distributed resources, and complex logic. It can be deployed across multiple platforms and inter-operate with many other applications, and it is long lived.
- **Business Oriented:** Its purpose is to meet specific business requirements. It encodes business policies, processes, rules, and entities, is developed in a business organization, and is deployed in a manner responsive to business needs.
- **Mission Critical:** An enterprise application must be robust enough to sustain continuous operation. It must be extremely flexible for scalability and deployment, and allow for efficient maintenance, monitoring, and administration

What is Scalability?

- The capacity to be changed in size or scale.
- Doing the same thing in a bigger way : more users, more domains etc.
- Vertical Scalability
 - ▶ Adding resource within the same logical unit to increase capacity. e.g.: Add more server resources
- Horizontal Scalability
 - ▶ Adding multiple logical units of resources and making them work as a single unit. Clustering Distributed systems etc.

N-Tier/Multi-Tier Architecture

N-tier architecture is also called multi-tier architecture because the software is engineered to have the processing, data management, and presentation functions physically and logically separated. That means that these different functions are hosted on several machines or clusters, ensuring that services are provided without resources being shared and, as such, these services are delivered at top capacity. The “N” in the name n-tier architecture refers to any number from 1

N-Tier/Multi-Tier Architecture

- Functionality of application separated into isolated functional areas called 'tiers'
- 3-tiered architecture:
 - ▶ **Client tier** - client program makes requests to middle tier
 - ▶ **Middle tier** - handles client requests by processing application data (business logic – business tier)
 - ▶ **Data tier** - persistent data store (database / legacy system / etc.)

Multi-Tiered Architecture

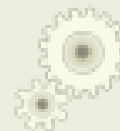
Presentation tier

The top-most level of the application is the user interface. The main function of the interface is to translate tasks and results to something the user can understand.

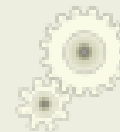


Logic tier

This layer coordinates the application, processes commands, makes logical decisions and evaluations, and performs calculations. It also moves and processes data between the two surrounding layers.



GET LIST OF ALL
SALES MADE
LAST YEAR



ADD ALL SALES
TOGETHER



Data tier

Here information is stored and retrieved from a database or file system. The information is then passed back to the logic tier for processing, and then eventually back to the user.



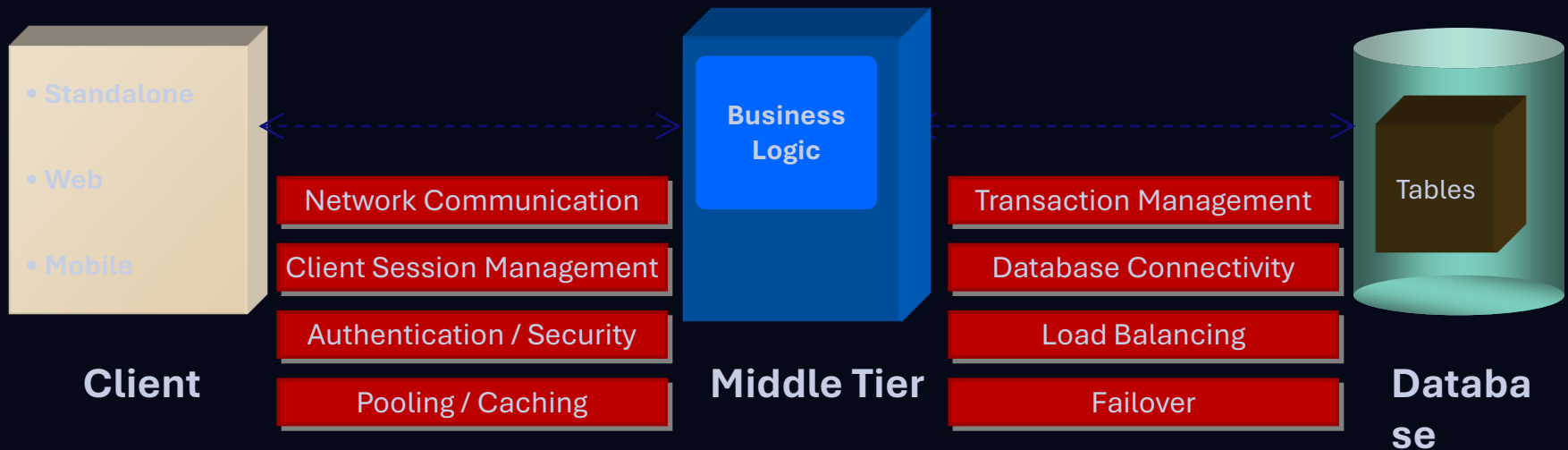
Database

Storage

Benefits of N-Tier Architecture

- **Secure:** You can secure each of the three tiers separately using different methods.
- **Easy to manage:** You can manage each tier separately, adding or modifying each tier without affecting the other tiers.
- **Scalable:** If you need to add more resources, you can do it per tier, without affecting the other tiers.
- **Flexible:** Apart from isolated scalability, you can also expand each tier in any manner that your requirements dictate.

Enterprise Applications



- In addition to business logic, we have to implement the other middle-tier services ('middleware')
 - ▶ Very Complex
 - ▶ Needs technical expertise
 - ▶ Time consuming
 - ▶ Very costly

Enterprise Applications

We do only this

**Business
Logic**



Java EE™

We don't worry about these

Network Communication

Client Session Management

Authentication / Security

Pooling / Caching

Transaction Management

Database Connectivity

Load Balancing

Failover

The Java Programming Language Platforms

There are four platforms of the Java programming language:

- Java Platform, Standard Edition (Java SE)
- Java Platform, Enterprise Edition (Java EE)
- Java Platform, Micro Edition (Java ME)
- JavaFX



Java Enterprise Edition

- The Java EE platform is built on top of the Java SE platform. The Java EE platform provides an API and runtime environment for developing and running large-scale, multi-tiered, scalable, reliable, and secure network applications.

Jakarta EE



Jakarta EE

- After 20 years Java EE was rebranded in September 2019
- Migrated from Oracle to Eclipse foundation under Eclipse Enterprise for Java EEJ4 project
- Renaming due to licensing and legal formalities – oracle owns the Java brand name
- High focus on the cloud
- New Era in Java Eco system
- Still an extension of Java EE 8

Java EE history

- J2EE 1.2 (December 12, 1999)
- J2EE 1.3 (September 24, 2001)
- J2EE 1.4 (November 11, 2003)
- Java EE 5 (May 11, 2006)
- Java EE 6 (December 10, 2009)
- Java EE 7 (May 28, 2013)
- Java EE 8 (August 31, 2017)
- Jakarta EE8 (September 10, 2019)

Java Technology – Java EE

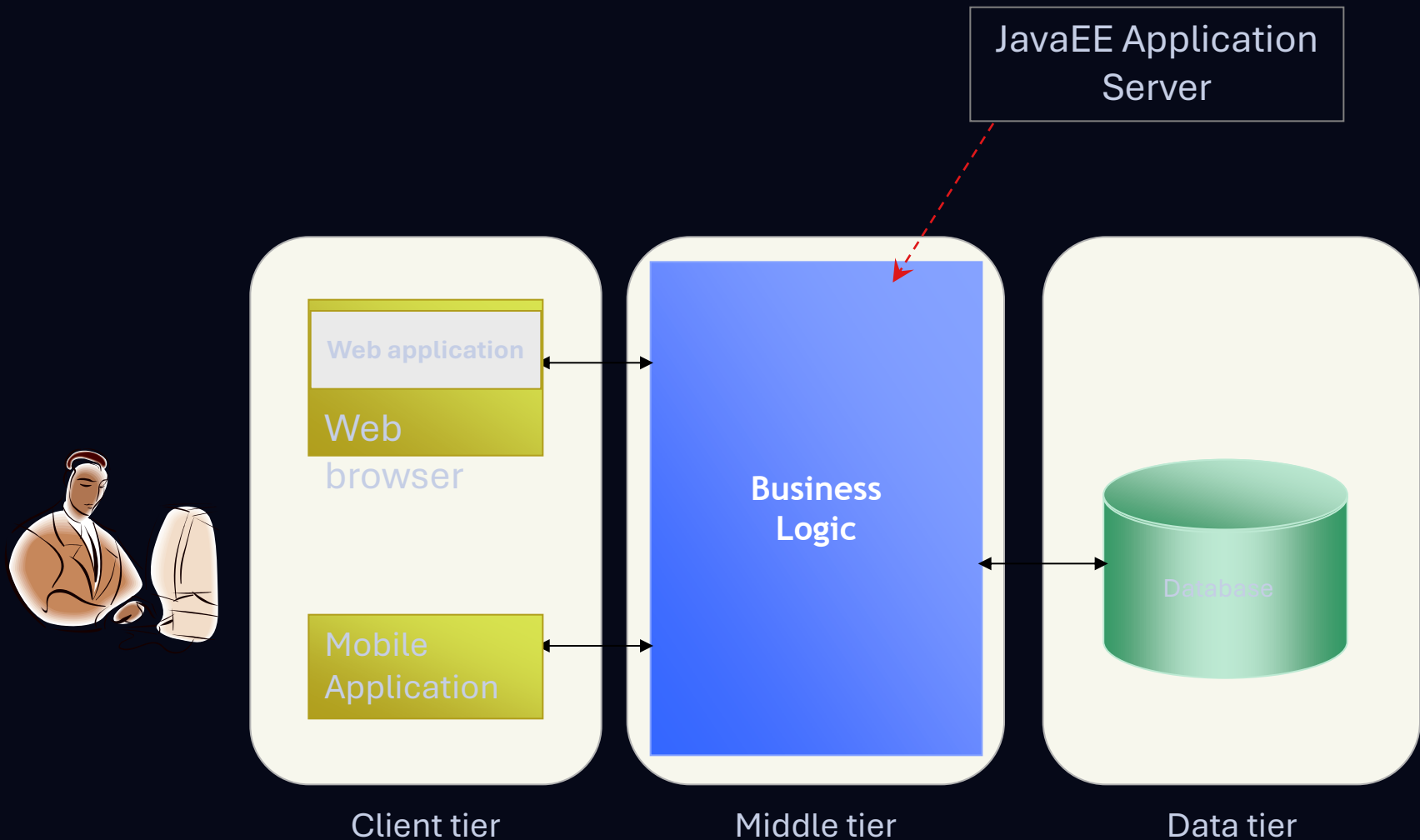
Java EE

Java SE

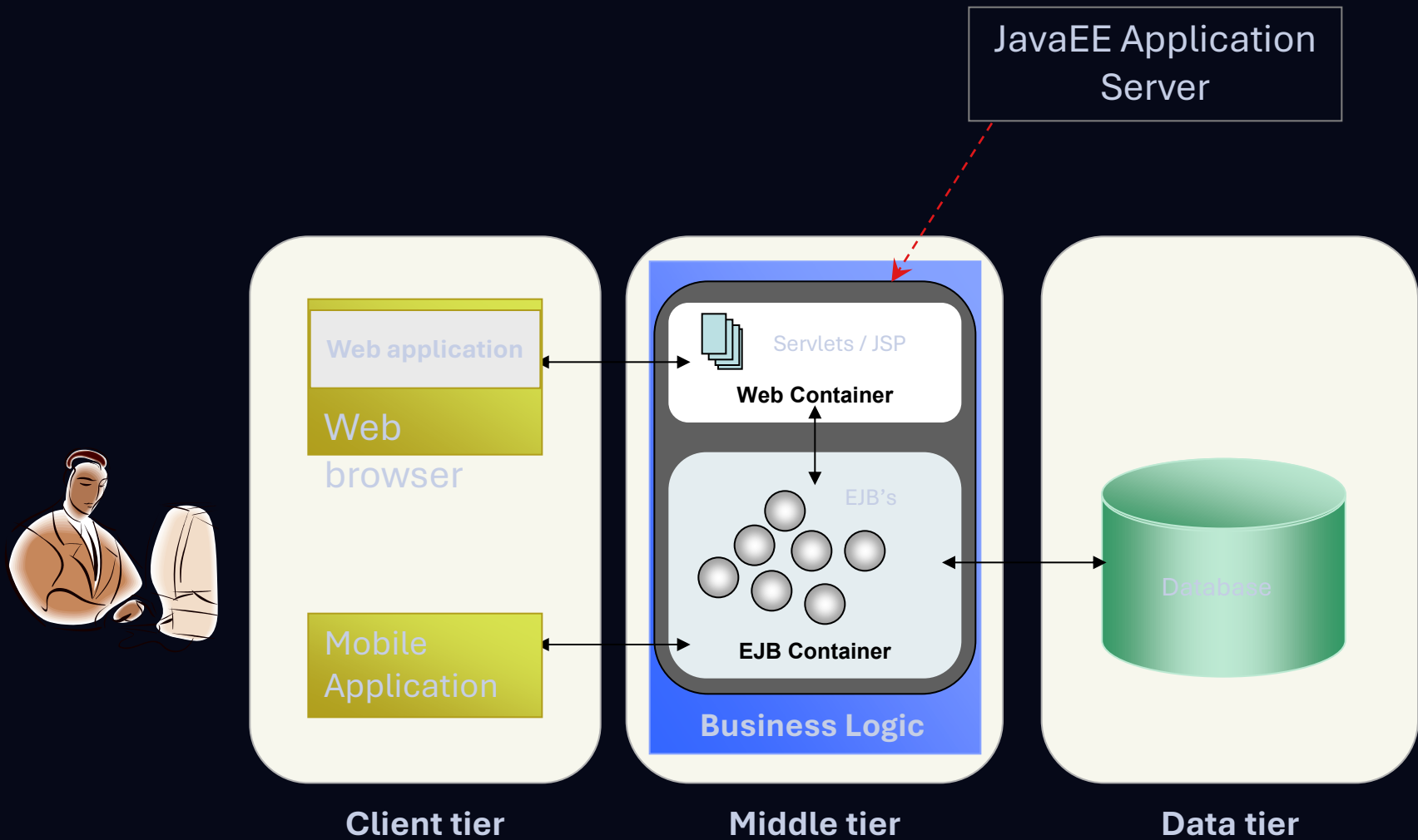
API For developing and running large-scale, multi-tiered, scalable, reliable and secure enterprise applications	Development Model Runtime environments (“containers”)
API Provides core functionality of Java ▪ Basic types and objects ▪ Input/output ▪ Networking ▪ Security ▪ GUI development ▪ Database access ▪ XML processing	Development tools, Deployment Technologies, other class libraries
JVM	

17

Multi-Tiered Applications



Multi-Tiered Applications...



EJB - Enterprise Java Beans

- An enterprise Java bean (EJB) is a server-side component that encapsulates the business logic of an application.
- The business logic is the code that fulfills the purpose of the application.
- Does not perform display of data (presentation) or perform database operations (persistence)

J2EE EJB Server / Container

- Manages the Enterprise beans contained within it self
- Responsibilities of the Server/Container:
 - ▶ Registering objects
 - ▶ Providing remote interfaces for the object
 - ▶ Creating and destroying object instances
 - ▶ Checking security for the object.
 - ▶ Managing active states for the object.
 - ▶ Co-ordinating distributed transactions.

Types of EJB's

- Session Beans
 - ▶ Stateful
 - ▶ Stateless
 - ▶ Singleton
- Message Driven Beans

Session beans

- Session bean encapsulates business logic only, it can be invoked by local, remote and webservice client.
- It can be used for calculations, database access etc.
- The life cycle of session bean is maintained by the application server (EJB Container).

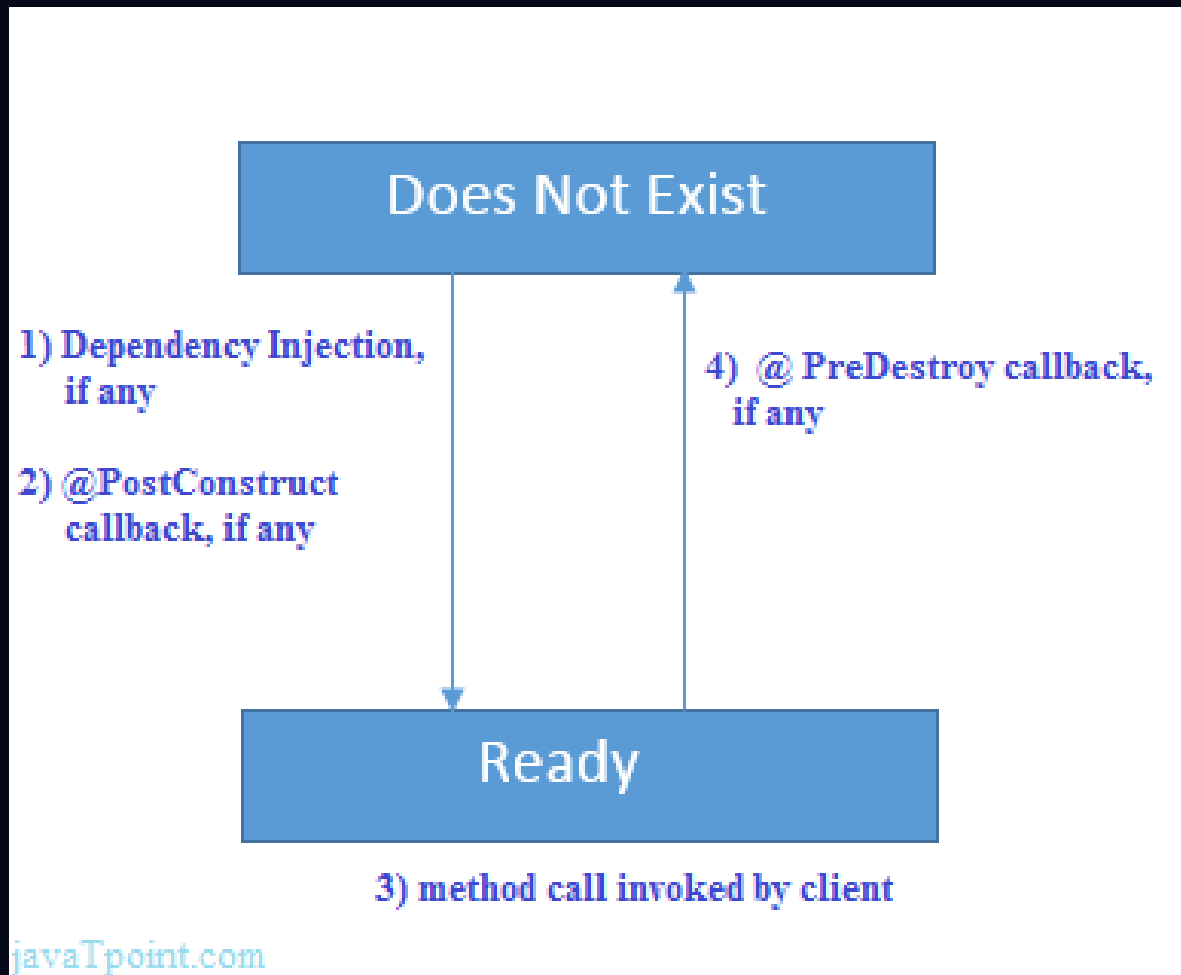
Types of Session Bean

- **1) Stateless Session Bean:** It doesn't maintain state of a client between multiple method calls.
- **2) Stateful Session Bean:** It maintains state of a client across multiple requests.
- **3) Singleton Session Bean:** One instance per application, it is shared between clients and supports concurrent access

Stateless Session Bean

- Stateless Session bean is a business object that represents business logic only. It doesn't have state (data).
- In other words, conversational state between multiple method calls is not maintained by the container in case of stateless session bean.
- The stateless bean objects are pooled by the EJB container to service the request on demand.
- It can be accessed by one client at a time. In case of concurrent access, EJB container routes each request to different instance.

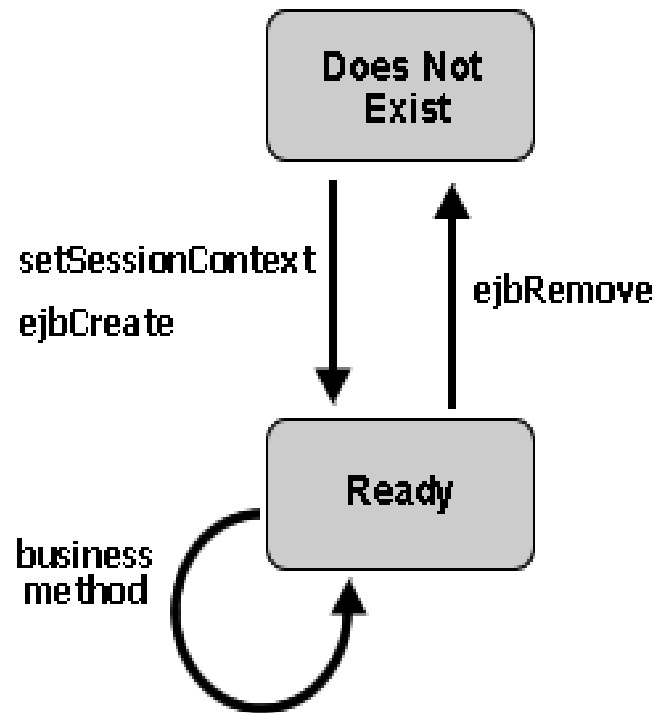
Stateless Session Bean



Stateful Session Bean

- **Stateful Session bean** *is a business object that represents business logic* like stateless session bean. But, it maintains state (data).
- A Stateful Session Bean maintains the conversational state with the client it is communicating.
- Each client creates a new instance of Stateful Bean and is not shared with other clients.
- When the communication between the client and bean ends, the Session Bean also terminates.

Stateful Session Bean



Singleton Session Bean

- A Singleton Session Bean maintains the state of the bean for the complete lifecycle of the application.
- Singleton Session Beans are similar to Stateless Session Beans but only one instance of the Singleton Session Bean is created in the whole application and does not terminates until the application is shut down.
- The single instance of the bean is shared between multiple clients and can be concurrently accessed.

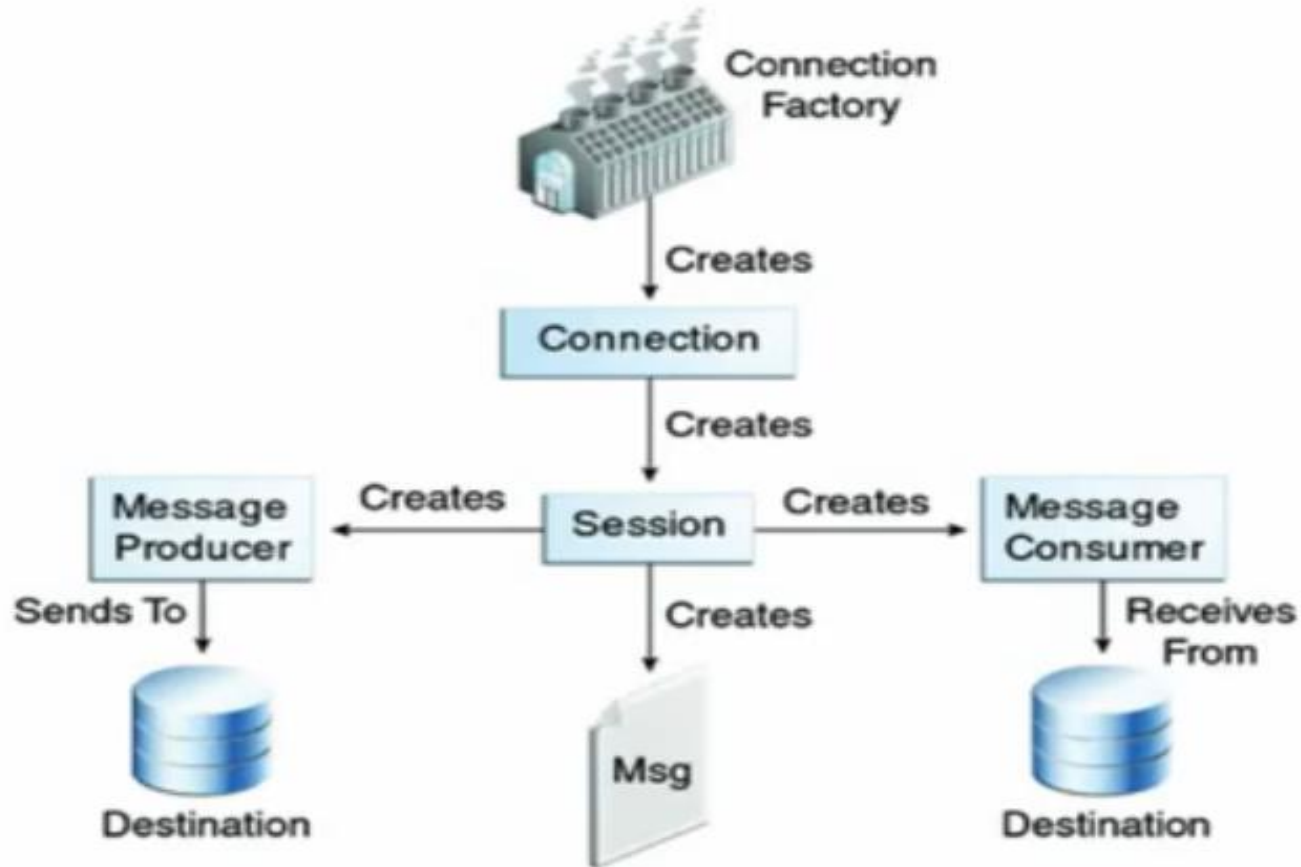
Message-Driven Bean

- An enterprise bean that allows Java EE applications to process messages asynchronously.
- Normally acts as a JMS message listener, which is similar to an event listener but receives JMS messages instead of events.
- Messages can be sent by any Java EE component (an application client, another enterprise bean, or a web component) or by a JMS application or system that does not use Java EE technology.

JMS(Java Messaging Service)

- An API that provides the facility to create, send and read messages. It provides loosely coupled, reliable and asynchronous communication.
- Advantages:
 - ▶ **Asynchronous:** To receive the message, client is not required to send request. Message will arrive automatically to the client.
 - ▶ **Reliable:** It provides assurance that message is delivered.
- Using JMS Applications communicate with each other using standard based messaging.

JMS Programming Model



More about JMS...

- Connection Factory: an object a client uses to create a connection to a provider. A connection factory encapsulates a set of connection-related parameters.
- JMS Destination : an object a client uses to specify the target of the messages. The two types of destinations in JMS are queues and topics.
- JMS connection : encapsulates a virtual connection with a JMS provider.
- JMS Sessions : a single threaded context for producing and consuming messages. Sessions are used to create – message producers, consumers, messages, queue browsers and temporary queues and topics.

More about JMS...

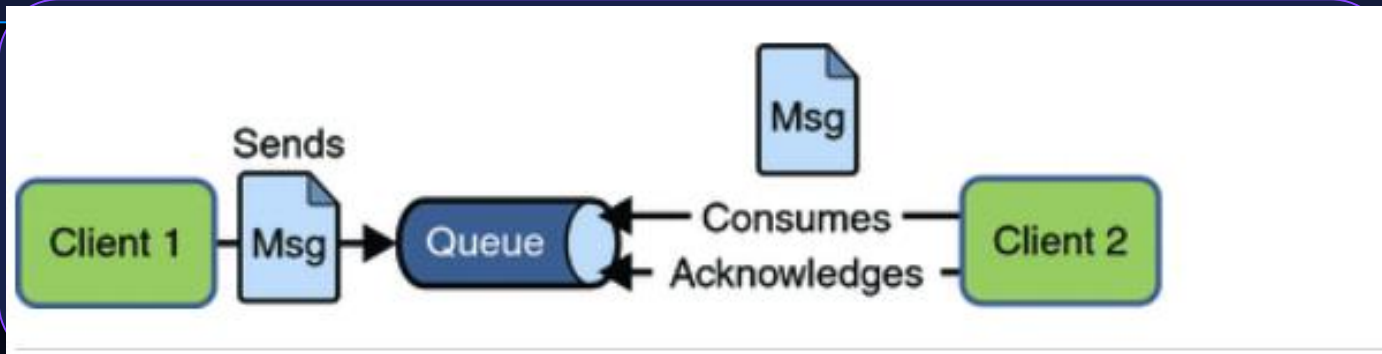
- JMS Message Listeners: an object that acts as an asynchronous event handler for messages. Implements the 'MessageListener' interface which contains the onMessage Method – which contains the logic that needs to be executed when a message arrives.

Messaging Domains

- Point-to-Point Messaging Domain
 - ▶ Queue
- Publisher/Subscriber Messaging Domain
 - ▶ Topic

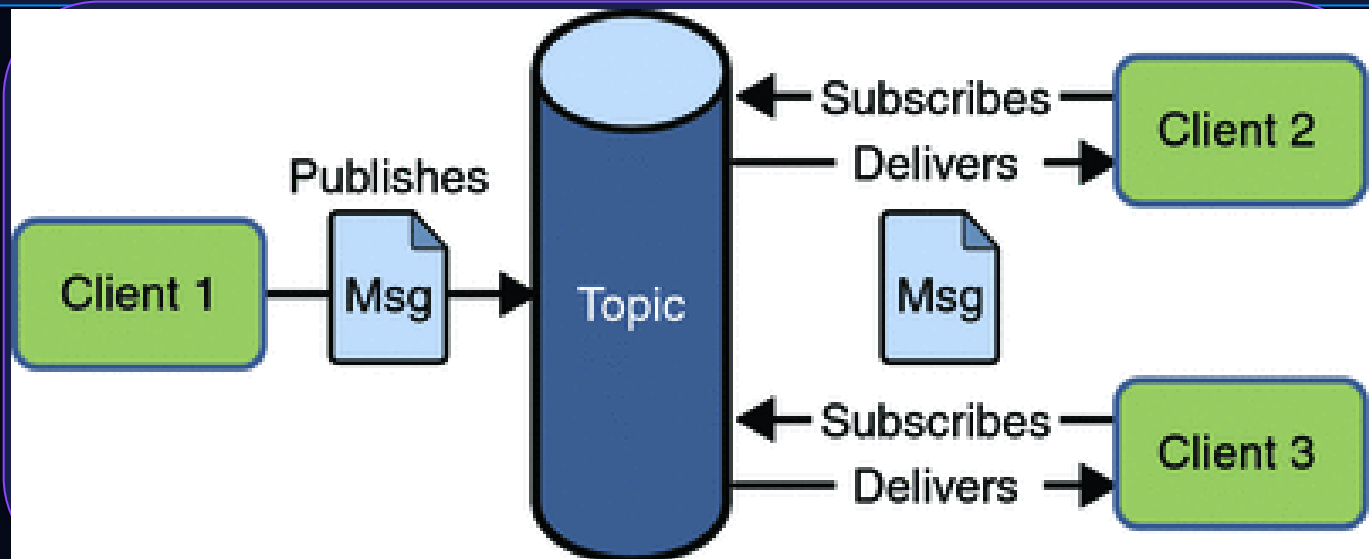
Point-to-Point (PTP) Messaging Domain

- In Ptp Model, One Message Is **Delivered To One Receiver** Only. Here, **Queue** Is Used As A Message Oriented Middleware (Mom).
- The Queue Is Responsible To Hold The Message Until Receiver Is Ready.
- In Ptp Model, There Is **No Timing Dependency** Between Sender And Receiver.



Publisher/Subscriber (Pub/Sub) Messaging Domain

- In Pub/Sub model, one message is **delivered to all the subscribers**. It is like broadcasting. Here, **Topic** is used as a message oriented middleware that is responsible to hold and deliver messages.
- In PTP model, there is **timing dependency** between publisher and subscriber.



REST Architecture

- **RE**presentational **S**tate **T**ransfer
- Software architectural style that defines the set of rules to be used for creating web services.
- Web services that conform to the REST architectural style, are called *RESTful* Web services,
- Uses HTTP protocol
- Allows variety of data formats
- RESTful web services are light weight, highly scalable and maintainable

RESTful Web Services - Java (JAX-RS)

- Java EE based set of API's for implementation of Restful web services
- JAX-RS uses annotations available from Java SE 5 to simplify the development creation and deployment of Restful Web Services

JAX-RS annotations

- **@Path**
 - ▶ Relative path of the resource class/method.
- **@GET**
 - ▶ HTTP Get request, used to fetch resource.
- **@PUT**
 - ▶ HTTP PUT request, used to update resource.
- **@POST**
 - ▶ HTTP POST request, used to create a new resource.

JAX-RS annotations

■ **@Produces**

- ▶ States the HTTP Response generated by web service. For example, APPLICATION/XML, TEXT/HTML, APPLICATION/JSON etc.

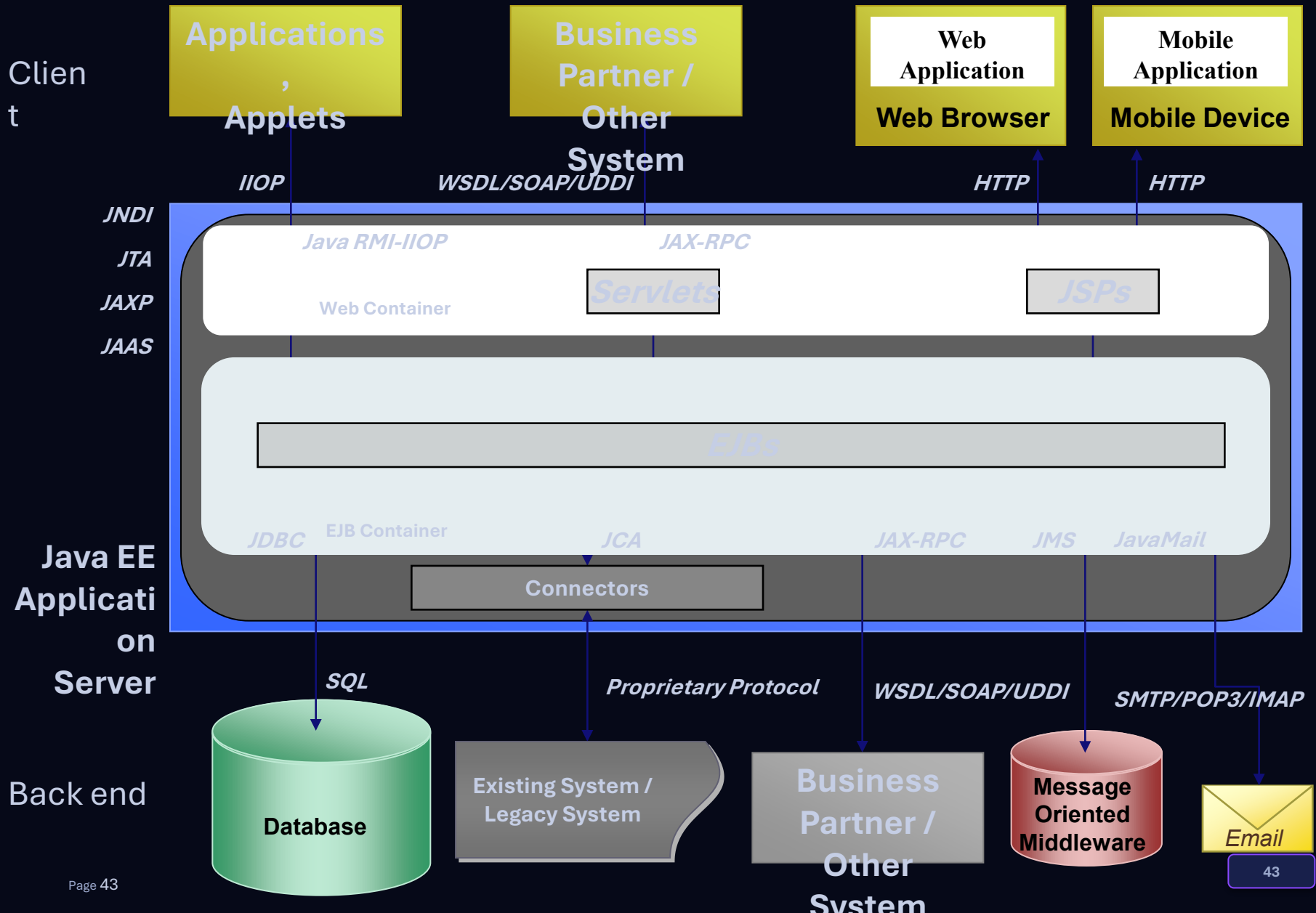
■ **@Consumes**

- ▶ States the HTTP Request type. For example, application/x-www-form-urlencoded to accept form data in HTTP body during POST request.

JAX-RS annotations

- **@PathParam**
 - ▶ Binds the parameter passed to the method to a value in path.
- **@QueryParam**
 - ▶ Binds the parameter passed to method to a query parameter in the path.
- **@FormParam**
 - ▶ Binds the parameter passed to the method to a form value.
- **@DefaultValue**
 - ▶ Assigns a default value to a parameter passed to the method.

Java EE Application Model + APIs



JNDI

- Java Naming and Directory Interface
- Utilized by JEE applications to locate resources and objects in portable fashion
 - Applications use symbolic names to find object references to resources via JNDI
 - The symbolic names and object references have to be configured by system administrator when the application is deployed.

What is a Servlet?

- Java™ objects which extend the functionality of a HTTP server
- Dynamic contents generation
- Better alternative to CGI, NSAPI, ISAPI, etc.
 - ▶ Efficient
 - ▶ Platform and server independent
 - ▶ Session management
 - ▶ Java-based

What is JSP Technology?

- Enables separation of business logic from presentation
 - ▶ Presentation is in the form of HTML or XML/XSLT
 - ▶ Business logic is implemented as Java Beans or custom tags
 - ▶ Better maintainability, reusability
- Extensible via custom tags
- Builds on Servlet technology

JEE API References

- **RMI-IIOP** (read as "RMI over IIOP") denotes the *Java remote method invocation* (RMI) interface over the *Internet Inter-Orb Protocol* (IIOP), which delivers *Common Object Request Broker Architecture* (CORBA) distributed computing capabilities to the Java 2 platform. Java RMI-IIOP was developed by Sun Microsystems and IBM, combining the best features of Java RMI technology with the best features of CORBA technology.
- The **Java Naming and Directory Interface (JNDI)** is a Java API for a directory service that allows Java software clients to discover and look up data and objects via a name. Like all Java APIs that interface with host systems, JNDI is independent of the underlying implementation. Additionally, it specifies a service provider interface (SPI) that allows directory service implementations to be plugged into the framework. The implementations may make use of a server, a flat file, or a database; the choice is up to the vendor.
- **JavaServer Pages (JSP)** is a Java technology that allows software developers to create dynamically generated web pages, with HTML, XML, or other document types, in response to a Web client request. The technology allows Java code and certain pre-defined actions to be embedded into static content.
- **Servlets** are Java programming language objects that dynamically process requests and construct responses. The **Java Servlet API** allows a software developer to add dynamic content to a Web server using the Java platform.

JEE API References...

- **Java API for XML-based RPC (JAX-RPC)** allows a Java application to invoke a Java-based Web Service with a known description while still being consistent with its WSDL description. It can be seen as Java RMI over Web services. JAX-RPC 2.0 was renamed to JAX-WS 2.0 (Java API for XML Web Services).
- **JDBC (Java Database Connectivity)** is an API for the Java programming language that defines how a client may access a database. It provides methods for querying and updating data in a database. JDBC is oriented towards relational databases.
- The **Java Transaction API (JTA)** is one of the Java EE APIs allowing distributed transactions to be done across multiple XA resources.
- The **Java API for XML Processing** or **JAXP** (pronounced jaks-p), is one of the Java XML programming APIs. It provides the capability of validating and parsing XML documents.
- **Java Authentication and Authorization Service**, or **JAAS**, pronounced "Jazz", is a Java security framework for user-centric security to augment the Java code-based security.

JEE API References...

- **Java EE Connector Architecture (JCA)** is a Java-based technology solution for connecting application servers and enterprise information systems (EIS) as part of enterprise application integration (EAI) solutions. While JDBC is specifically used to connect Java EE applications to databases, JCA is a more generic architecture for connection to legacy systems (including databases).
- The **Java Message Service (JMS)** API is a Java Message Oriented Middleware (MOM) API for sending messages between two or more clients.
- **JavaMail** is a Java API used to receive and send email via SMTP,POP3 and IMAP.

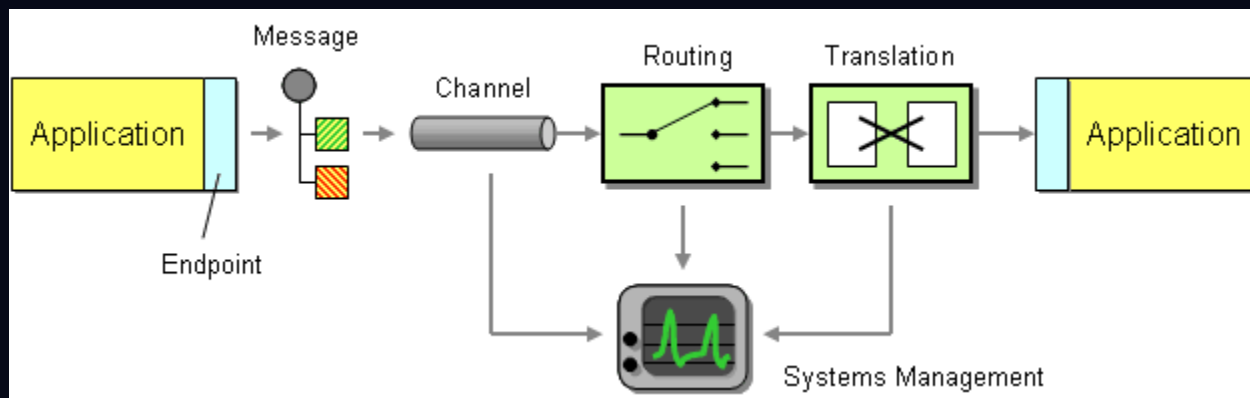
Enterprise Application Integration

- The use of software and computer systems' architectural principles to integrate a set of enterprise computer applications
- Why?
 - ▶ Enterprises are typically comprised of hundreds if not thousands of applications that are custom-built, acquired from a third-party, part of a legacy system, or a combination thereof, operating in multiple tiers of different operating system platforms. It is not uncommon to find an enterprise that has 30 different Websites, three instances of the same ERP and countless departmental solutions
 - ▶ Creating a single, big application to run a complete business is next to impossible.
 - ▶ Spreading business functions across multiple applications provides the business with the flexibility to select the “best” system that best suits the business’ needs

Common types of Ent.App. Integration

- Information Portals
- Data Replication
- Shared Business Functions
- Service-Oriented Architectures
- Distributed Business Processes
- Business-to-Business Integration

Basic Elements of an Integration Solution



Questions?

- Feel free to drop a mail to sachit.talagala@ifsworld.com
- Questions like
 - ▶ Paper structure
 - ▶ Weight of marks for mid semester and final exam

Should be directed to your course co-Ordinator 😊